TRACK	FLAVOUR	ENVIRONMENT	LIBRARIES
2 · closed-ended vault, Lending Protocol V1.1	Loaded · XLS-47 Oracle, XLS-70 Credentials, XLS-80 Permissioned Domains, XLS-85 Token Escrow, MPT shares	XRPL Devnet · wss://s.devnet.rippletest.net:51233 · network_id 2 · rippled 3.4.0-rc5 (rc4 when we started)	xrpl 5.2.0 · ripple-binary-codec 2.11.0 · Node 24.13.0 · frontend xrpl-connect 0.8.2 · DevEx tender-gopher-21, keen-auk-43

The brief pins xrpl.js@5.2.0-beta.0; stable 5.2.0 shipped 8 h 40 min before kickoff and we pinned it. Every claim here is evidenced with transaction hashes in feedback/appendix.pdf: 40 findings, 7 P0, 19 P1, 14 P2, 3 withdrawn.

0 What the ledger would not tell us

Forty-eight hours reading a closed-ended vault honestly, for the two people who ran the workshop.

We built Orma, a reader that grades closed-ended lending vaults from the ledger, publishes the grade as a native XLS-47 oracle and lets it gate deposits through a Permissioned Domain. The protocol turned out to be well designed and hard to read: its rules are right, but too many of them live in C++ comments, unmerged pull requests and result codes that carry four meanings.

The design deserves praise first. A closed-ended vault forbids maturity transformation at the ledger level: `VaultWithdraw` is refused for the whole Investment phase, and no `LoanSet` may mature after `RedemptionDate`. Maple and Goldfinch retrofitted lockups only after a run, Maple with 54 million dollars of sour debt on its books; XLS-66 made the run mechanism illegal. We built with coding agents, and re-ran every claim below on Devnet.

THE FIRST HOUR

The toolchain moved under us first. For seventeen days before the event `npm install xrpl` gave 5.1.0, which could not serialise a closed-ended `VaultCreate`; the failure surfaced as a codec error inside `Wallet.sign()` after `validate()` had passed it. Version 5.2.0 fixed that on the night of 11 September, so two of our findings became release notes before we could file them, the best fate a finding can have. Python has no such luck: no published `xrpl-py` can counterparty-sign a `LoanSet`.

Then the wall. `LoanBrokerSet` on a vault we owned returned `tecNO_PERMISSION`, though the documentation says only that the submitter must be the owner. Two hours went on key derivation and field shapes before we read `LoanBrokerSet.cpp`, where a comment says that only a closed-ended vault may host a broker. `VaultKind`, `SubscriptionDate` and `RedemptionDate` appear zero times in the published XLS-65 and XLS-66; that specification is pull request 587, open since 21 July. rippled computes the reason and discards it. The C++ comments were the most accurate documentation that exists.

This is why the documented first hour cannot complete. The lending tutorial dies at `Setting up tutorial: 5/7` with `TypeError: Cannot read properties of undefined`: `lendingSetup.js` creates an open-ended vault, `LoanBrokerSet` returns `tecNO_PERMISSION`, `submitAndWait` does not throw on a `tec`, and a `.find()` over the metadata returns `undefined`. The developer sees a stack trace, not a result code, a vault kind or an amendment name. Our fix creates the vault closed-ended, checks results before indexing `AffectedNodes`, polls `close_time` before `LoanSet` and bumps the pinned `xrpl ^4.6.0`. The corrected chain runs green in 79.6 seconds on rc5 and is ready to file.

D2 · documentation/tutorials · P0 · xrpl-dev-portal, xrpl ^4.6.0 · contrib/tutorial-fix/ · D1, L1, L2

In your workshop you remarked that doing with a closed vault what belongs to an open one should give a no-permission result rather than an error. On the lending path that result is the confusing one, because it carries four meanings: not closed-ended, or not the owner, on `LoanBrokerSet`; maturing after redemption, on `LoanSet`; already impaired, on `LoanManage`. At 17:31 on Saturday a `LoanSet` that should have succeeded came back with it, and only the term told us why.

U1 · UX · P1 · rippled 3.4.0-rc5 · src/dev/check-loanset-overload.mjs · Appendix E

THE EMPTY CHANGE SET

Shota gave us the case that makes a wrong reading dangerous. Bob deposits 100 RLUSD into vault A, receives 100 shares, and pledges them to a different broker, who lends to him from vault B; Bob loops the position the way people loop on Aave. Broker B must value the shares, and the obvious valuation is `AssetsTotal / shares`, which one-to-one issuance invites. After an impairment in vault A the honest one is `(AssetsTotal − LossUnrealized) / shares`. Broker B is under-collateralised, cannot tell, and the loop multiplies the error.

Here is what happened when we tried to catch that moment. On the first `LoanManage` with `tfLoanImpair` against a healthy vault, the Vault node's `PreviousFields` is empty while `FinalFields` carries a non-zero `LossUnrealized`. `LossUnrealized` is not stored on a healthy vault at all, so it has no previous value for the change set to record; under cash-basis accounting an impairment changes nothing else on the Vault, so an indexer that diffs metadata sees a node touched but unchanged. The same transaction does record the Loan's previous `Flags` of 0, which is the distinction: what governs the change set is whether the field was there, not whether it was zero. We captured it twice on rc5, at 18:06 and 23:07 on Saturday: `AssetsTotal` 51,000,000 to 51,000,000, `AssetsAvailable` 41,000,000 to 41,000,000, `LossUnrealized` 0 to 10,000,000. The naive value reads 1.000000 and the correct one 0.803922, 1,961 basis points apart; the reported figure overstates by 19.61 per cent of itself. The trap is asymmetric, which is why it survives testing: unimpairment and a second impairment both diff correctly. It cost us nothing, because Orma re-reads the objects rather than diffing metadata, and would have cost anyone else the vault. The prior art: `rippled#6487`, opened on 5 March by the operator of the XRPLWin explorer, describes the identical behaviour and was closed by its reporter the same day as "Not a bug." No maintainer replied; what is missing is one paragraph saying that an empty `PreviousFields` does not mean unchanged.

`X1 · missing primitive · P0 · rippled 3.4.0-rc5, any client · tx 075FE6D2…49229C3 · §5`

Maxime came to us with an idea: put a pointer to an honest valuation inside the share's own `MPTokenMetadata`, so it travels with the token. We built it that evening. The metadata persists when set at `VaultCreate`, but `VaultSet` cannot update it and `MPTokenIssuanceSet` by the owner returns `tecNO_PERMISSION`, because the issuer is the vault pseudo-account. The channel is write-once forever and settable only on the transaction that creates the vault, so the metadata can never name the object it describes; our pointer is a template the reader completes from the token it holds. rippled warns there that it is not XLS-89 conformant, the only reason we knew XLS-89 existed, on the one transaction after which nothing can be fixed.

`Share metadata, write-once · documentation/tutorials · P1 · demo/metadata-vault.json · Appendix D1–D3`

THE HISTORY THAT LIES, AND THE MATHS NOBODY EVALUATED

We found the next one by building. On Saturday around 18:20 we baked a vault whose only credit event was one impairment, and our broker track record showed one event where we had submitted two. `LoanManage` carries a `LoanID` and no `LoanBrokerID`, and an impairment does not touch the `LoanBroker` object at all, so a filter over `account_tx` drops every impairment and keeps defaults, which do move `DebtTotal`. Nothing errored, and the consequence inverted our intent: our conduct score counts impairments to decide whether a loss was signalled before it was taken, so an honest manager who flagged a loan early was penalised twenty points, and the output looked plausible. Twenty-five minutes to find, and it would have shipped silently. The join is free once you know to look: `Loan.FinalFields.LoanBrokerID` is in the same metadata.

`M5 · a broker's history cannot be reconstructed · missing primitive · P1 · rc5 · tx 9CFF971F…D14106 · src/history.mjs`

When Alex showed Shota the default-ordering result, he already knew the maths, and asked how hard the first-loss arithmetic had been to find and to understand. Easy to find, thirty minutes too slow to understand, and he asked us to write here that a business example with real numbers beside the formula would have saved those minutes. Cover liquidated on a default is capped by `ceil(DebtTotal × CoverRateMinimum/1e5 × CoverRateLiquidation/1e5)`, and `DebtTotal` is the broker's whole book at that instant, so each default shrinks the base for the next. On the facility we called Kestrel, two loans of 30 and 10 XRP defaulted largest-first consumed 0.50 XRP of first-loss capital; smallest-first would have consumed

0.70. Same losses, forty per cent more cover absorbed in the order fairer to depositors, and only the broker owner chooses. The one worked example in the specification is a single isolated default, exactly the case where order cannot matter. An unevaluated rule reads as a rule with no consequences.

D11 · documentation/tutorials · P1 · XLS-66 §3.10.5, eq. 35 · demo/ordering-vault.json · U3, L9

SIX AMENDMENTS IN ONE FLOW

The gate worked at 22:55 on Saturday: a graded depositor's `VaultDeposit` succeeded, an ungraded one returned `tecNO_AUTH`, and after revocation the deposit still failed while withdrawal succeeded, so entry closes and exit stays open, which is right. `tecNO_AUTH` was the most precise code we met all weekend. Two traps. `DomainID` is not on the Vault entry but on the share issuance, so the obvious lookup returns nothing, which reads exactly like open to everyone. And our first run proved its own test wrong: in a 25-second window the revocation steps returned `tecEXPIRED`, because the phase rule preempts the credential rule, which is never evaluated. A correct code, wholly misleading about what was tested.

C1 · documentation/tutorials · P1 · rc5 · demo/gate-vault.json · Appendix C1

Three more, briefly. Delegation does not compose with lending: granting `Payment` succeeded as our control, and every lending transaction returned `temMALFORMED`, with nothing to say the suite is out of scope. The oracle was the cleanest composition, since `get_aggregate_price` computes the median across publishers inside the ledger, though `OracleSet` is not a merge and a reused document fills with ghosts. And a vault whose asset is another vault's share returned `tecWRONG_ASSET`, with the reason only in `VaultCreate.cpp`, though people close to the protocol read the specification as permitting it. Escrow of shares works and puts Shota's pledge on the ledger. Here we owe a retraction: the same escrow appears in both parties' owner directories, we summed both, doubled the collateral, briefly reported a missing index, and withdrew it the same day.

Two last frictions. Each lending scenario cost four to six minutes of waiting, and immutable dates make every vault single-use, so nobody will write a test suite against this. And `xrpl-connect` 0.8.2 imports `xrpl` without declaring it, and ships no type declarations despite a "Type Safe" badge. That badge annoyed me most: a package that promises types and ships none costs more than one that promises nothing.

C4, C5 · UX · P1 · tx 547DA719…10AF17 · xrpl-connect 0.8.2 · client libraries · P2 · U8, U2

YOUR SIX QUESTIONS

The three-phase model is coherent once learned and nowhere written: "investment phase" matches nothing in either specification. `VaultKind`, `SubscriptionDate` and `RedemptionDate` are self-explanatory as names, absent from the published standards, and omitted by xrpl.js 5.2.0's `VaultInfoResponse`. The phase codes are sensible, though `tecTOO_SOON` during Investment reads like a hiccup rather than a lock until redemption, and the final-payment constraint was not clear before trial. On the explorer the first impairment looks unchanged to anything that diffs metadata, and a broker's impairments vanish from `account_tx`. Cash-basis accounting behaved as the Track 2 note says, and is exactly what makes the change set empty.

WHAT WE WOULD FIX, WHAT WE GIVE BACK, WHAT WE GOT WRONG

First, one paragraph on the metadata page saying that `PreviousFields` omits fields that were absent before. Then the tutorial's `VaultCreate` and result check, with 3923, 587 and 582 merged. Then a broker's actions made attributable, and the reason rippled already computes returned with the code. What worked: the closed-ended design, `tecNO_AUTH`, the revocation asymmetry, aggregation inside the ledger, escrow of shares, the C++ comments, and a release that fixed two of our bugs before we could report them. We give back the tutorial fix, verified green on rc5, and the register of forty findings with hashes. Filed on 13 September: rippled#8222, xrpl.js#3477 and four comments; the tutorial was not, as 3923 has four of its five fixes. We got four things wrong: a probe that misread the permission code, the escrow claim we retracted, a headline we marked new that 6487 had described first, and an SDK we said did not exist. A report you cannot catch being wrong is worth less than one that shows its corrections.

Alexandre Lemiere & Andrea Gonzalez

Paris, 13 September 2026 · @frytegg · @andreagnzz · Team BuzzBallz

Corrected 13 October 2026: PreviousFields restated as presence, not value (xrpl-dev-portal#3926).